Isolation levels for Sakila Dataset

Airon Matthew Bantillo
airon_bantillo@dlsu.edu.ph
De La Salle University
Manila, Philippines

Bullet Andre Fernandez
bullet_fernandez@dlsu.edu.ph
De La Salle University
Manila, Philippines

Charles Sebastian V. Infante
charles_infante@dlsu.edu.ph
De La Salle University
Manila, Philippines

Alec Marx G. Nono
alec_nono@dlsu.edu.ph
De La Salle University
Manila, Philippines

Jean Rondel Ponce
jean_ponce@dlsu.edu.ph
De La Salle University
Manila, Philippines

1 Introduction

Modern relational database management systems (RDBMS) rely on concurrency controls to enable the simultaneous execution of multiple transactions while ensuring data integrity. ANSI's SQL-92 standards define four transaction isolation levels to balance data consistency and control a system's susceptibility to the following three read anomalies: **Dirty Reads**, **Non-Repeatable Reads**, and **Phantom Reads**.

This study evaluates the operation behavior of MySQL's **InnoDB** storage engine under concurrent workload execution. A multi-session testing environment was constructed to observe Multi-Version Concurrency Control behavior across all four isolation levels: **READ UNCOMMITTED**, **READ COMMITTED**, **REPEATABLE READ**, and **SERIALIZABLE**.

1.1 Relational Schema Design

The experimental domain models MySQL's Sakila database. The relational tables are defined with explicit foreign key constraints and transactional support powered by the **InnoDB** engine.

The schema definition for the tracking tables is implemented as follows:

Listing 1: DDL Schema Definition for Accounts and Transactions

```
CREATE TABLE IF NOT EXISTS accounts (
    account_id INT PRIMARY KEY AUTO_INCREMENT,
    customer_id INT NOT NULL,
    balance DECIMAL(10, 2) NOT NULL DEFAULT 0.00,
    status VARCHAR(20) NOT NULL DEFAULT 'ACTIVE',
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON
     UPDATE CURRENT_TIMESTAMP,
    FOREIGN KEY (customer_id) REFERENCES
     customer(customer_id)
) ENGINE=InnoDB;

CREATE TABLE IF NOT EXISTS transactions (
    transaction_id INT PRIMARY KEY AUTO_INCREMENT,
    account_id INT NOT NULL,
    type VARCHAR(20) NOT NULL, -- 'DEPOSIT', 'WITHDRAWAL',
     'TRANSFER'
    amount DECIMAL(10, 2) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (account_id) REFERENCES accounts(account_id)
) ENGINE=InnoDB;
```

1.2 Initial Seed Data

The environment was initialized with three core account entities to provide baseline states for concurrent query execution:

Accounts Initial State

101	1	1000.00	ACTIVE
102	2	500.00	ACTIVE
103	3	2500.00	ACTIVE

Transactions Initial State

101	DEPOSIT	1000.00
102	DEPOSIT	500.00
103	DEPOSIT	2500.00

2 Experiments

To evaluate concurrency control across the four ANSI SQL-92 isolation levels (**READ UNCOMMITTED**, **READ COMMITTED**, **REPEATABLE READ**, and **SERIALIZABLE**), a multi-session experimental framework was implemented. The test harness employs two concurrent database connections managed by a multithreaded Python harness with barrier synchronization (`threading.Barrier`), enforcing precise statement interleaving across dual sessions via controlled `DO SLEEP()` delays.

A total of 12 baseline experiments (4 isolation levels×3 read anomalies) were systematically executed and logged. Prior to every individual test case, the database state was reset and re-seeded using automated DDL/DML scripts to ensure deterministic initial conditions.

2.1 Experimental SQL Transactions

The transactional workloads were designed to isolate the three classical read phenomena. In accordance with SQL coding conventions, transactional commands, query targets, and timing delays are presented below.

2.1.1 Dirty Read Workload. A transaction updates Account 101 without committing, while a concurrent transaction reads the uncommitted balance before a rollback occurs.

Listing 2: Session 1 (Writer) Transaction for Dirty Read

```
-- Session 1: Writer
SET SESSION TRANSACTION ISOLATION LEVEL <LEVEL>;
START TRANSACTION;
UPDATE accounts SET balance = balance - 300.00 WHERE
     account_id = 101;
```

```
SELECT 'W after update' AS step, balance FROM accounts
     WHERE account_id = 101;
DO SLEEP(6); -- Hold uncommitted change
ROLLBACK;    -- Reverts update back to 1000.00
SELECT 'W after rollback' AS step, balance FROM accounts
     WHERE account_id = 101;
```

Listing 3: Session 2 (Reader) Transaction for Dirty Read

```
-- Session 2: Reader
SET SESSION TRANSACTION ISOLATION LEVEL <LEVEL>;
DO SLEEP(2); -- Wait for Session 1's UPDATE
START TRANSACTION;
SELECT 'R1' AS step, balance FROM accounts WHERE account_id
     = 101;
DO SLEEP(6); -- Wait for Session 1's ROLLBACK
SELECT 'R2' AS step, balance FROM accounts WHERE account_id
     = 101;
COMMIT;
```

2.1.2 Non-Repeatable Read Workload. Session 1 performs an initial read on Account 101, pauses, and re-reads the row. Concurrently, Session 2 modifies and commits the balance during the pause.

Listing 4: Session 1 (Reader) Transaction for Non-Repeatable Read

```
-- Session 1: Reader
SET SESSION TRANSACTION ISOLATION LEVEL <LEVEL>;
START TRANSACTION;
SELECT 'R1' AS step, balance FROM accounts WHERE account_id
     = 101;
DO SLEEP(6); -- Pause while Session 2 updates and commits
SELECT 'R2' AS step, balance FROM accounts WHERE account_id
     = 101;
COMMIT;
```

Listing 5: Session 2 (Writer) Transaction for Non-Repeatable Read

```
-- Session 2: Writer
SET SESSION TRANSACTION ISOLATION LEVEL <LEVEL>;
DO SLEEP(2); -- Wait for Session 1's first read
START TRANSACTION;
UPDATE accounts SET balance = balance + 200.00 WHERE
     account_id = 101;
COMMIT;
SELECT 'W committed' AS step, balance FROM accounts WHERE
     account_id = 101;
```

2.1.3 Phantom Read Workload. Session 1 executes a range query on accounts with `balance >= 800.00`, pauses, and re-executes the query. Session 2 concurrently inserts a new qualifying row (Account 104 with balance $1500.00). In Session 1, an update statement is also executed to evaluate InnoDB current-read behavior under REPEATABLE READ.

Listing 6: Session 1 (Reader) Transaction for Phantom Read

```
-- Session 1: Reader
SET SESSION TRANSACTION ISOLATION LEVEL <LEVEL>;
SET SQL_SAFE_UPDATES = 0;
START TRANSACTION;
SELECT 'P1' AS step, account_id, balance FROM accounts
     WHERE balance >= 800.00 ORDER BY account_id;
DO SLEEP(6); -- Pause while Session 2 inserts row 104
SELECT 'P2' AS step, account_id, balance FROM accounts
     WHERE balance >= 800.00 ORDER BY account_id;
UPDATE accounts SET status = 'FLAGGED' WHERE balance >=
     800.00;
SELECT 'P3' AS step, account_id, balance FROM accounts
     WHERE balance >= 800.00 ORDER BY account_id;
COMMIT;
```

Listing 7: Session 2 (Inserter) Transaction for Phantom Read

```
-- Session 2: Inserter
SET SESSION TRANSACTION ISOLATION LEVEL <LEVEL>;
DO SLEEP(2); -- Wait for Session 1's first range read
START TRANSACTION;
INSERT INTO accounts (account_id, customer_id, balance,
     status) VALUES (104, 4, 1500.00, 'ACTIVE');
COMMIT;
```

2.2 Summary of Baseline Results

Table 1 summarizes the empirical findings across all 12 baseline test cases, comparing ANSI-92 expectations with observed InnoDB behavior.

2.3 Two-Column Interleaved Timelines (12 Baseline Cases)

The transaction interleaving for all 12 baseline configurations is presented below. Timestamps (t_x) denote elapsed wall-clock execution time captured by the automated runner.

2.3.1 Dirty Read Interleaving (Cases 1–4). Tables 2 and 3 document the concurrent execution between Session 1 (Writer) and Session 2 (Reader).

2.3.2 Non-Repeatable Read Interleaving (Cases 5–8). Tables 4 and 5 document the concurrent execution between Session 1 (Reader) and Session 2 (Writer).

2.3.3 Phantom Read Interleaving (Cases 9–12). Tables 6 and 7 document the concurrent execution between Session 1 (Reader) and Session 2 (Inserter).

2.4 Explicit Locking Experiments Framework (Member 4)

To evaluate lock-based concurrency control as an alternative to elevating entire transaction isolation levels, four explicit locking configurations were tested using row-level locks:

(1) **Lock Case 1 (Preventing Dirty Reads under READ UNCOMMITTED):** Using `SELECT ... FOR UPDATE` or table locks.
(2) **Lock Case 2 (Preventing Non-Repeatable Reads under READ COMMITTED):** Employing `SELECT ... FOR SHARE` or `FOR UPDATE` on single-row lookups.

Table 1: Comprehensive 12-Case Baseline Concurrency Matrix on MySQL InnoDB

Case	Anomaly	Isolation Level	ANSI-92	InnoDB Observed	Match	Observed Behavioral Note
1	Dirty Read	READ UNCOMMITTED	Allowed	**ANOMALY**	Yes	S2 reads uncommitted balance ($700.00) at t = 2.0s prior to S1 rollback.
2	Dirty Read	READ COMMITTED	Prevented	None	Yes	S2 reads committed pre-image ($1000.00) from undo log.
3	Dirty Read	REPEATABLE READ	Prevented	None	Yes	S2 reads from consistent snapshot ($1000.00).
4	Dirty Read	SERIALIZABLE	Prevented	None	Yes	S2 read is blocked for 4.0s on S1's Exclusive Lock until rollback.
5	Non-Repeatable Read	READ UNCOMMITTED	Allowed	**ANOMALY**	Yes	S1 reads $1000.00 at t = 0s and $1200.00 at t = 6s ($R_1 \neq R_2$).
6	Non-Repeatable Read	READ COMMITTED	Allowed	**ANOMALY**	Yes	Second query establishes fresh Read View, reflecting committed $1200.00.
7	Non-Repeatable Read	REPEATABLE READ	Prevented	None	Yes	Transaction-level Read View preserves $1000.00 pre-image.
8	Non-Repeatable Read	SERIALIZABLE	Prevented	None	Yes	S1's Shared Lock blocks S2's UPDATE for 4.0s until S1 commits.
9	Phantom Read	READ UNCOMMITTED	Allowed	**ANOMALY**	Yes	Newly inserted Account 104 appears on second range query (P_2: 3 rows).
10	Phantom Read	READ COMMITTED	Allowed	**ANOMALY**	Yes	Fresh statement Read View reveals newly committed Account 104.
11	Phantom Read	REPEATABLE READ	Allowed	**Write-Only**	Yes	Snapshot hides Account 104 in P_2, but range UPDATE touches 104, revealing it in P_3.
12	Phantom Read	SERIALIZABLE	Prevented	None	Yes	S1's Next-Key Locks block S2's INSERT for 4.0s until S1 commits.

Table 2: Two-Column Interleaved Timelines: Dirty Read under READ UNCOMMITTED and READ COMMITTED

Time (t_x)	Session 1: Writer (`account_id = 101`)	Session 2: Reader (`account_id = 101`)
Case 1: READ UNCOMMITTED — Verdict: ANOMALY OBSERVED		
t = 0.0s	START TRANSACTION; UPDATE accounts SET balance = balance - 300.00 WHERE account_id = 101;	DO SLEEP(2);
t = 0.0s	SELECT 'W after update', balance → **700.00**; DO SLEEP(6);	(waiting on sleep)
t = 2.0s	(holding uncommitted update)	START TRANSACTION; SELECT 'R1', balance → **700.00 [DIRTY READ]**;
t = 6.0s	ROLLBACK; SELECT 'W after rollback', balance → **1000.00**;	DO SLEEP(6);
t = 8.0s	(transaction finished)	SELECT 'R2', balance → **1000.00**; COMMIT;
Case 2: READ COMMITTED — Verdict: PREVENTED		
t = 0.0s	START TRANSACTION; UPDATE accounts SET balance = balance - 300.00 WHERE account_id = 101;	DO SLEEP(2);
t = 0.0s	SELECT 'W after update', balance → **700.00**; DO SLEEP(6);	(waiting on sleep)
t = 2.0s	(holding uncommitted update)	START TRANSACTION; SELECT 'R1', balance → **1000.00 [NO DIRTY READ]**;
t = 6.0s	ROLLBACK; SELECT 'W after rollback', balance → **1000.00**;	DO SLEEP(6);
t = 8.0s	(transaction finished)	SELECT 'R2', balance → **1000.00**; COMMIT;

(3) **Lock Case 3 (Preventing Phantom Reads under READ COMMITTED / REPEATABLE READ):** Employing locking range reads (`SELECT ... FOR UPDATE WHERE balance >= 800.00`) to invoke explicit Next-Key locking.
(4) **Lock Case 4 (Performance and Deadlock Benchmarks under Explicit Locking):** Measuring lock-wait latency, lock wait timeouts (`innodb_lock_wait_timeout`), and throughput trade-offs across concurrent connections.

References

[1] Abraham Silberschatz, Henry F. Korth, and S. Sudarshan. 2020. *Database System Concepts* (7th ed.). McGraw-Hill.
[2] Sakila Database Reference Archive. 2025. MySQL Documentation Team.
[3] Patricia G. Selinger, et al. 1979. Access Path Selection in a Relational Database Management System. In *Proceedings of the 1979 ACM SIGMOD International Conference on Management of Data*. 23–34.

Table 3: Two-Column Interleaved Timelines: Dirty Read under REPEATABLE READ and SERIALIZABLE

Time (t_x)	Session 1: Writer (`account_id = 101`)	Session 2: Reader (`account_id = 101`)
Case 3: REPEATABLE READ — Verdict: PREVENTED		
t = 0.0s	START TRANSACTION; UPDATE accounts SET balance = balance - 300.00 WHERE account_id = 101;	DO SLEEP(2);
t = 0.0s	SELECT 'W after update', balance → **700.00**; DO SLEEP(6);	(waiting on sleep)
t = 2.0s	(holding uncommitted update)	START TRANSACTION; SELECT 'R1', balance → **1000.00 [NO DIRTY READ]**;
t = 6.0s	ROLLBACK; SELECT 'W after rollback', balance → **1000.00**;	DO SLEEP(6);
t = 8.0s	(transaction finished)	SELECT 'R2', balance → **1000.00**; COMMIT;
Case 4: SERIALIZABLE — Verdict: PREVENTED VIA LOCK CONTENTION		
t = 0.0s	START TRANSACTION; UPDATE accounts SET balance = balance - 300.00 WHERE account_id = 101; [Holds X-Lock]	DO SLEEP(2);
t = 0.0s	SELECT 'W after update', balance → **700.00**; DO SLEEP(6);	(waiting on sleep)
t = 2.0s	(holding Exclusive X-Lock)	START TRANSACTION; SELECT 'R1', balance → **BLOCKED** (waits for S-Lock);
t = 6.0s	ROLLBACK; [Releases X-Lock]; SELECT 'W after rollback' → **1000.00**;	**UNBLOCKED** (waited 4.0s) → SELECT 'R1', balance → **1000.00**; DO SLEEP(6);
t = 12.0s	(transaction finished)	SELECT 'R2', balance → **1000.00**; COMMIT;

Table 4: Two-Column Interleaved Timelines: Non-Repeatable Read under READ UNCOMMITTED and READ COMMITTED

Time (t_x)	Session 1: Reader (`account_id = 101`)	Session 2: Writer (`account_id = 101`)
Case 5: READ UNCOMMITTED — Verdict: ANOMALY OBSERVED		
t = 0.0s	START TRANSACTION; SELECT 'R1', balance → **1000.00**; DO SLEEP(6);	DO SLEEP(2);
t = 2.0s	(holding transaction open)	START TRANSACTION; UPDATE accounts SET balance = balance + 200.00 WHERE account_id = 101; COMMIT;
t = 2.0s	(holding transaction open)	SELECT 'W committed', balance → **1200.00**;
t = 6.0s	SELECT 'R2', balance → **1200.00 [NON-REPEATABLE READ:** 1000 ≠ 1200**]**; COMMIT;	(transaction finished)
Case 6: READ COMMITTED — Verdict: ANOMALY OBSERVED		
t = 0.0s	START TRANSACTION; SELECT 'R1', balance → **1000.00**; DO SLEEP(6);	DO SLEEP(2);
t = 2.0s	(holding transaction open)	START TRANSACTION; UPDATE accounts SET balance = balance + 200.00 WHERE account_id = 101; COMMIT;
t = 2.0s	(holding transaction open)	SELECT 'W committed', balance → **1200.00**;
t = 6.0s	SELECT 'R2', balance → **1200.00 [NON-REPEATABLE READ: fresh Read View]**; COMMIT;	(transaction finished)

Table 5: Two-Column Interleaved Timelines: Non-Repeatable Read under REPEATABLE READ and SERIALIZABLE

Time (t_x)	Session 1: Reader (`account_id = 101`)	Session 2: Writer (`account_id = 101`)
Case 7: REPEATABLE READ — Verdict: PREVENTED		
t = 0.0s	START TRANSACTION; SELECT 'R1', balance → **1000.00** [Read View created]; DO SLEEP(6);	DO SLEEP(2);
t = 2.0s	(holding transaction open)	START TRANSACTION; UPDATE accounts SET balance = balance + 200.00 WHERE account_id = 101; COMMIT;
t = 2.0s	(holding transaction open)	SELECT 'W committed', balance → **1200.00**;
t = 6.0s	SELECT 'R2', balance → **1000.00 [REPEATABLE READ: Read View reused]**; COMMIT;	(transaction finished)
Case 8: SERIALIZABLE — Verdict: PREVENTED VIA LOCK CONTENTION		
t = 0.0s	START TRANSACTION; SELECT 'R1', balance → **1000.00** [Acquires S-Lock]; DO SLEEP(6);	DO SLEEP(2);
t = 2.0s	(holding Shared S-Lock)	START TRANSACTION; UPDATE accounts SET balance = balance + 200.00 → **BLOCKED**;
t = 6.0s	SELECT 'R2', balance → **1000.00**; COMMIT; [Releases S-Lock];	**UNBLOCKED** (waited 4.0s) → UPDATE completes; COMMIT;
t = 6.0s	(transaction finished)	SELECT 'W committed', balance → **1200.00**;

Table 6: Two-Column Interleaved Timelines: Phantom Read under READ UNCOMMITTED and READ COMMITTED

Time (t_x)	Session 1: Reader (`WHERE balance >= 800.00`)	Session 2: Inserter (`account_id = 104`)
Case 9: READ UNCOMMITTED — Verdict: ANOMALY OBSERVED		
t = 0.0s	START TRANSACTION; SELECT 'P1', account_id → **[101, 103]** (2 rows); DO SLEEP(6);	DO SLEEP(2);
t = 2.0s	(holding transaction open)	START TRANSACTION; INSERT INTO accounts VALUES (104, 4, 1500.00, 'ACTIVE'); COMMIT;
t = 6.0s	SELECT 'P2', account_id → **[101, 103, 104]** (3 rows) **[PHANTOM READ]**;	(transaction finished)
t = 6.0s	UPDATE accounts SET status = 'FLAGGED' WHERE balance >= 800.00;	(transaction finished)
t = 6.0s	SELECT 'P3', account_id → **[101, 103, 104]**; COMMIT;	(transaction finished)
Case 10: READ COMMITTED — Verdict: ANOMALY OBSERVED		
t = 0.0s	START TRANSACTION; SELECT 'P1', account_id → **[101, 103]** (2 rows); DO SLEEP(6);	DO SLEEP(2);
t = 2.0s	(holding transaction open)	START TRANSACTION; INSERT INTO accounts VALUES (104, 4, 1500.00, 'ACTIVE'); COMMIT;
t = 6.0s	SELECT 'P2', account_id → **[101, 103, 104]** (3 rows) **[PHANTOM READ: fresh Read View]**;	(transaction finished)
t = 6.0s	UPDATE accounts SET status = 'FLAGGED' WHERE balance >= 800.00;	(transaction finished)
t = 6.0s	SELECT 'P3', account_id → **[101, 103, 104]**; COMMIT;	(transaction finished)

Table 7: Two-Column Interleaved Timelines: Phantom Read under REPEATABLE READ and SERIALIZABLE

Time (t_x)	Session 1: Reader (`WHERE balance >= 800.00`)	Session 2: Inserter (`account_id = 104`)
Case 11: REPEATABLE READ — Verdict: WRITE-ONLY PHANTOM OCCURS		
t = 0.0s	START TRANSACTION; SELECT 'P1', account_id → **[101, 103]** (2 rows) [Snapshot created]; DO SLEEP(6);	DO SLEEP(2);
t = 2.0s	(holding transaction open)	START TRANSACTION; INSERT INTO accounts VALUES (104, 4, 1500.00, 'ACTIVE'); COMMIT;
t = 6.0s	SELECT 'P2', account_id → **[101, 103]** (2 rows) **[HIDDEN BY MVCC SNAPSHOT]**;	(transaction finished)
t = 6.0s	UPDATE accounts SET status = 'FLAGGED' WHERE balance >= 800.00; [Current-read updates row 104];	(transaction finished)
t = 6.0s	SELECT 'P3', account_id → **[101, 103, 104]** (3 rows) **[WRITE-ONLY PHANTOM REVEALED]**; COMMIT;	(transaction finished)
Case 12: SERIALIZABLE — Verdict: PREVENTED VIA NEXT-KEY GAP LOCK		
t = 0.0s	START TRANSACTION; SELECT 'P1', account_id → **[101, 103]** (2 rows) [Holds Next-Key Locks]; DO SLEEP(6);	DO SLEEP(2);
t = 2.0s	(holding Next-Key Locks on range and gap)	START TRANSACTION; INSERT INTO accounts VALUES (104, 4, 1500.00, 'ACTIVE') → **BLOCKED**;
t = 6.0s	SELECT 'P2', account_id → **[101, 103]** (2 rows);	(waiting on Next-Key lock)
t = 6.0s	UPDATE accounts SET status = 'FLAGGED' WHERE balance >= 800.00; (affected = 2 rows);	(waiting on Next-Key lock)
t = 6.0s	SELECT 'P3', account_id → **[101, 103]** (2 rows); COMMIT; [Releases Next-Key Locks];	**UNBLOCKED** (waited 4.0s) → INSERT completes; COMMIT;